NAME:HAFSA
HT NO: 2303A52381
BATCH 39

AI ASSISTED CODING-12.2

```python
# A national disaster management authority wants to develop a Smart Disaster Relief Resource Management System to efficiently manage relief camps, victims, and resource distribution during natural disasters such as floods or earthquakes. You are required to design and implement a Python application using object-oriented programming (classes and constructors), loops, conditional statements, file handling, and basic data analysis.The system should manage multiple relief camps. Each camp has a camp ID, location, maximum capacity, available food packets, medical kits, and volunteers. When a disaster victim arrives, the system should register the victim by storing details such as victim ID, name, age, health condition (normal/critical), and assigned camp. The system must automatically check camp capacity before assigning a victim. If the camp is full, it should display an appropriate message.The program should also allow distribution of food and medical kits to victims. If a victim is marked as "critical," the system should prioritize medical kit allocation. Resource quantities must be updated automatically after distribution.All camp details and victim records must be stored permanently in files. The administrator should be able to perform operations repeatedly such as adding a new camp, registering victims, distributing resources, viewing records, searching for a victim by ID, and generating reports.The final analytical report should display:Total number of camps,Total victims registered,Camp with highest occupancy,Total food packets distributed,Total medical kits distributed,Number of critical victims
import json
import os
CAMP_FILE = "camps.json"
VICTIM_FILE = "victims.json"
REPORT_FILE = "report.json"
def load_data(filename):
    if os.path.exists(filename):
        with open(filename, "r") as f:
            return json.load(f)
    return {}
def save_data(filename, data):
    with open(filename, "w") as f:
        json.dump(data, f, indent=4)
class Camp:
    def __init__(self, camp_id, location, capacity, food, medical, volunteers):
        self.camp_id = camp_id
        self.location = location
        self.capacity = capacity
        self.food = food
        self.medical = medical
        self.volunteers = volunteers
        self.occupancy = 0
class Victim:
    def __init__(self, victim_id, name, age, health, camp_id):
        self.victim_id = victim_id
        self.name = name
        self.age = age
        self.health = health
        self.camp_id = camp_id
        self.food_received = 0
        self.medical_received = 0
def add_camp():
    camps = load_data(CAMP_FILE)
    camp_id = input("Enter Camp ID: ")
    location = input("Enter Location: ")
    capacity = int(input("Enter Maximum Capacity: "))
    food = int(input("Enter Available Food Packets: "))
    medical = int(input("Enter Available Medical Kits: "))
```

Ln 167, Col 5 Spaces: 4 UTF-8 CRLF {} Python

```python
def add_camp():
    volunteers = int(input("Enter Number of Volunteers: "))
    new_camp = Camp(camp_id, location, capacity, food, medical, volunteers)
    camps[camp_id] = vars(new_camp)
    save_data(CAMP_FILE, camps)
    print("Camp added successfully!\n")
def register_victim():
    camps = load_data(CAMP_FILE)
    victims = load_data(VICTIM_FILE)
    victim_id = input("Enter Victim ID: ")
    name = input("Enter Name: ")
    age = int(input("Enter Age: "))
    health = input("Enter Health Condition (normal/critical): ").lower()
    camp_id = input("Enter Camp ID to assign: ")
    if camp_id not in camps:
        print("Camp not found!\n")
        return
    if camps[camp_id]["occupancy"] >= camps[camp_id]["capacity"]:
        print("Camp is full! Cannot assign victim.\n")
        return
    new_victim = Victim(victim_id, name, age, health, camp_id)
    victims[victim_id] = vars(new_victim)
    camps[camp_id]["occupancy"] += 1
    save_data(VICTIM_FILE, victims)
    save_data(CAMP_FILE, camps)
    print("Victim registered successfully!\n")
def distribute_resources():
    camps = load_data(CAMP_FILE)
    victims = load_data(VICTIM_FILE)
    report = load_data(REPORT_FILE)
    victim_id = input("Enter Victim ID: ")
    if victim_id not in victims:
        print("Victim not found!\n")
        return
    victim = victims[victim_id]
    camp = camps[victim["camp_id"]]
    # Food Distribution
    if camp["food"] > 0:
        camp["food"] -= 1
        victim["food_received"] += 1
        report["food_distributed"] = report.get("food_distributed", 0) + 1
        print("Food packet distributed.")
    else:
        print("No food packets available.")
    # Medical Distribution (Priority to critical)
    if victim["health"] == "critical":
        if camp["medical"] > 0:
```

A12.py > ...

```python
def distribute_resources():
            if camp["medical"] > 0:
                camp["medical"] -= 1
                victim["medical_received"] += 1
                report["medical_distributed"] = report.get("medical_distributed", 0) + 1
                print("Medical kit distributed (PRIORITY).")
            else:
                print("No medical kits available.")
        else:
            print("Medical kit not required.")
        save_data(CAMP_FILE, camps)
        save_data(VICTIM_FILE, victims)
        save_data(REPORT_FILE, report)
    def view_records():
        camps = load_data(CAMP_FILE)
        victims = load_data(VICTIM_FILE)
        print("\n--- CAMPS ---")
        for camp in camps.values():
            print(camp)
        print("\n--- VICTIMS ---")
        for victim in victims.values():
            print(victim)
        print()
    def search_victim():
        victims = load_data(VICTIM_FILE)
        victim_id = input("Enter Victim ID to search: ")
        if victim_id in victims:
            print(victims[victim_id])
        else:
            print("Victim not found.")
        print()
    def generate_report():
        camps = load_data(CAMP_FILE)
        victims = load_data(VICTIM_FILE)
        report = load_data(REPORT_FILE)
        total_camps = len(camps)
        total_victims = len(victims)
        highest_occupancy = 0
        busiest_camp = None
        for camp in camps.values():
            if camp["occupancy"] > highest_occupancy:
                highest_occupancy = camp["occupancy"]
                busiest_camp = camp["camp_id"]

        critical_count = sum(1 for v in victims.values() if v["health"] == "critical")
        print("\n----- ANALYTICAL REPORT -----")
        print("Total Camps:", total_camps)
```

```python
def generate_report():
    print("Total Victims Registered:", total_victims)
    print("Camp with Highest Occupancy:", busiest_camp)
    print("Total Food Packets Distributed:", report.get("food_distributed", 0))
    print("Total Medical Kits Distributed:", report.get("medical_distributed", 0))
    print("Number of Critical Victims:", critical_count)
    print("-------------------------------\n")
def main():
    while True:
        print("SMART DISASTER RELIEF RESOURCE MANAGEMENT SYSTEM")
        print("1. Add Camp")
        print("2. Register Victim")
        print("3. Distribute Resources")
        print("4. View Records")
        print("5. Search Victim")
        print("6. Generate Report")
        print("7. Exit")
        choice = input("Enter choice: ")
        if choice == "1":
            add_camp()
        elif choice == "2":
            register_victim()
        elif choice == "3":
            distribute_resources()
        elif choice == "4":
            view_records()
        elif choice == "5":
            search_victim()
        elif choice == "6":
            generate_report()
        elif choice == "7":
            print("Exiting system...")
            break
        else:
            print("Invalid choice!\n")
if __name__ == "__main__":
    main()

# Task Description -1 (Data Structures - Stack Implementation with AI Assistance
```

```python

# Task Description -1 (Data Structures – Stack Implementation with AI Assistance
# ➢ Task: Use AI assistance to generate a Python program that
# implements a Stack data structure.
# Instructions:
# Prompt AI to create a Stack class with the following methods:
# ➢ push(element)
# ➢ pop()
# ➢ peek()
# ➢ is_empty()
# ➢ Ensure proper error handling for stack underflow.
# ➢ Ask AI to include clear docstrings for each method.
# Expected Output:
# ➢ A functional Python program implementing a Stack using a
# class.
# ➢ Properly documented methods with docstrings.
class Stack:
    def __init__(self):
        self.items = []
    def push(self, element):
        self.items.append(element)
    def pop(self):
        if self.is_empty():
            raise IndexError("Stack Underflow: Cannot pop from empty stack.")
        return self.items.pop()
    def peek(self):
        if self.is_empty():
            raise IndexError("Stack is empty.")
        return self.items[-1]
    def is_empty(self):
        return len(self.items) == 0
# Testing
if __name__ == "__main__":
    s = Stack()
    s.push(10)
    s.push(20)
    print("Top element:", s.peek())
    print("Popped:", s.pop())
    print("Is empty?", s.is_empty())
```

```python
    print("Is empty:", s.is_empty())
# Task Description -2 (Algorithms – Linear vs Binary Search
# Analysis)
# ➢ Task: Use AI to implement and compare Linear Search and
# Binary Search algorithms in Python.
# Instructions:
# ➢ Prompt AI to generate:
# ➢ linear_search(arr, target)
# ➢ binary_search(arr, target)
# ➢ Include docstrings explaining:
# ➢ Working principle
# ➢ Test both algorithms using different input sizes.
# Expected Output:
# • Python implementations of both search algorithms.
# • AI-generated comments and complexity analysis.
# • Test results showing correctness and comparison.
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
# Testing
if __name__ == "__main__":
    arr = list(range(1, 10001))  # Large input
    print("Linear Search Result:", linear_search(arr, 9999))
    print("Binary Search Result:", binary_search(arr, 9999))
```

```python

# Task Description -3 (Test Driven Development – Simple Calculator
# Function)
# ➢ Task:
# Apply Test Driven Development (TDD) using AI assistance to
# develop a calculator function.
# Instructions:
# ➢ Prompt AI to first generate unit test cases for addition and
# subtraction.
# ➢ Run the tests and observe failures.
# ➢ Ask AI to implement the calculator functions to pass all tests.
# ➢ Re-run the tests to confirm success.
# Expected Output:
# ➢ Separate test file and implementation file.
# ➢ Test cases executed before implementation.
# ➢ Final implementation passing all test cases.
'''import unittest
from calculator import add, subtract
class TestCalculator(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(5, 3), 8)
        self.assertEqual(add(-1, 1), 0)
    def test_subtract(self):
        self.assertEqual(subtract(5, 3), 2)
        self.assertEqual(subtract(0, 5), -5)
if __name__ == "__main__":
    unittest.main()'''
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
```

```python
# Task Description -4 (Data Structures – Queue Implementation with
# AI Assistance)
# ➢ Task:
# Use AI assistance to generate a Python program that implements
# a Queue data structure.
# Instructions:
# ➢ Prompt AI to create a Queue class with the following methods:
# • enqueue(element)
# • dequeue()
# • front()
# • is_empty()
# ➢ Handle queue overflow and underflow conditions.
# ➢ Include appropriate docstrings for all methods.
# Expected Output:
# ➢ A fully functional Queue implementation in Python.
# ➢ Proper error handling and documentation.
class Queue:
    def __init__(self, capacity):
        self.queue = []
        self.capacity = capacity
    def enqueue(self, element):
        if len(self.queue) >= self.capacity:
            raise OverflowError("Queue Overflow")
        self.queue.append(element)
    def dequeue(self):
        if self.is_empty():
            raise IndexError("Queue Underflow")
        return self.queue.pop(0)
    def front(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self.queue[0]
    def is_empty(self):
        """Return True if queue is empty."""
        return len(self.queue) == 0
# Testing
if __name__ == "__main__":
    q = Queue(3)
    q.enqueue(10)
    q.enqueue(20)
    print("Front:", q.front())
    print("Dequeued:", q.dequeue())
```

```python
# Task Description -5 (Algorithms – Bubble Sort vs Selection Sort)
# ➢ Task:
# Use AI to implement Bubble Sort and Selection Sort algorithms
# and compare their behavior.
# Instructions:
# ➢ Prompt AI to generate:
# • bubble_sort(arr)
# • selection_sort(arr)
# ➢ Include comments explaining each step.
# ➢ Add docstrings mentioning time and space complexity.
# Expected Output:
# • Correct Python implementations of both sorting algorithms.
# • Complexity analysis in docstrings.
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_index = i
        for j in range(i+1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr
# Testing
if __name__ == "__main__":
    arr = [64, 25, 12, 22, 11]
    print("Bubble Sort:", bubble_sort(arr.copy()))
    print("Selection Sort:", selection_sort(arr.copy()))
```

```
''' ALGORITHMS -

#  Task 1: Stack - Algorithm

## 🔹 Algorithm for `push(element)`
1. Start
2. Check if stack exists
3. Insert the element at the top of the stack
4. End
---

## 🔹 Algorithm for `pop()`
1. Start
2. Check if stack is empty
3. If empty → Display "Stack Underflow" and Stop
4. Else remove the top element
5. Return the removed element
6. End
---

## 🔹 Algorithm for `peek()`
1. Start
2. Check if stack is empty
3. If empty → Display error message
4. Else return the top element
5. End
---

## 🔹 Algorithm for `is_empty()`
1. Start
2. If size of stack = 0 → Return True
3. Else return False
4. End
---
```

```
# Task 2: Searching Algorithms
## 🔹 Algorithm for Linear Search
1. Start
2. Input array and target element
3. For each element in array
   * If element equals target
     * Return index
4. If loop ends and target not found
   * Return -1
5. End
## 🔹 Algorithm for Binary Search
(Precondition: Array must be sorted)
1. Start
2. Set left = 0
3. Set right = length of array - 1
4. While left ≤ right
   * mid = (left + right) // 2
   * If arr[mid] == target → Return mid
   * If arr[mid] < target → left = mid + 1
   * Else → right = mid - 1
5. If not found → Return -1
6. End
---

# Task 3: Calculator using TDD
## 🔹 Algorithm for Addition Function
1. Start
2. Input numbers a and b
3. Compute sum = a + b
4. Return sum
5. End
## 🔹 Algorithm for Subtraction Function
1. Start
2. Input numbers a and b
3. Compute difference = a - b
4. Return difference
5. End
## 🔹 Algorithm for Unit Testing
1. Start
2. Import calculator functions
3. Define test cases
4. Call function with sample inputs
5. Compare expected and actual output
6. If equal → Test Passed
7. Else → Test Failed
8. End
```

```
---
# Task 4: Queue Algorithm
## 🔹 Algorithm for `enqueue(element)`
1. Start
2. Check if queue is full
3. If full → Display "Queue Overflow"
4. Else insert element at rear
5. End
## 🔹 Algorithm for `dequeue()`
1. Start
2. Check if queue is empty
3. If empty → Display "Queue Underflow"
4. Else remove front element
5. Return removed element
6. End
## 🔹 Algorithm for `front()`
1. Start
2. If queue is empty → Display error
3. Else return front element
4. End
## 🔹 Algorithm for `is_empty()`
1. Start
2. If queue size = 0 → Return True
3. Else return False
4. End
# Task 5: Sorting Algorithms
## 🔹 Algorithm for Bubble Sort
1. Start
2. Input array of size n
3. For i = 0 to n-1
    * For j = 0 to n-i-2
        * If arr[j] > arr[j+1]
            * Swap arr[j] and arr[j+1]
4. Return sorted array
5. End
## 🔹 Algorithm for Selection Sort
1. Start
2. Input array of size n
3. For i = 0 to n-1
    * Set min_index = i
    * For j = i+1 to n-1
        * If arr[j] < arr[min_index]
            * Update min_index
    * Swap arr[i] and arr[min_index]
4. Return sorted array
5. End
```

ANALYSIS:

Task 1: Stack Implementation – Analysis

The Stack follows the LIFO (Last In First Out) principle and provides constant time O(1) operations for push, pop, and peek. Proper underflow handling ensures reliability. It is useful in applications like expression evaluation and recursion.

Task 2: Linear vs Binary Search – Analysis

Linear Search works on both sorted and unsorted data but has O(n) time complexity. Binary Search is more efficient with O(log n) time complexity but requires sorted data. Binary Search is preferred for large datasets.

Task 3: Test Driven Development (Calculator) – Analysis

TDD improves code reliability by writing test cases before implementation. It ensures correctness and reduces debugging time. The calculator functions operate in constant time O(1).

Task 4: Queue Implementation – Analysis

The Queue follows the FIFO (First In First Out) principle. Enqueue is O(1), while dequeue is O(n) when using a list. Proper overflow and underflow handling improves robustness.

Task 5: Bubble Sort vs Selection Sort – Analysis

Both algorithms have $O(n^2)$ time complexity, making them inefficient for large datasets. Bubble Sort is stable and adaptive, while Selection Sort performs fewer swaps. They are mainly suitable for small datasets.

OUTPUT:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\hp\Desktop\AIAC> python A12.py
SMART DISASTER RELIEF RESOURCE MANAGEMENT SYSTEM
1. Add Camp
2. Register Victim
3. Distribute Resources
4. View Records
5. Search Victim
6. Generate Report
7. Exit
Enter choice: 1
Enter Camp ID: 123
Enter Location: hnk
Enter Maximum Capacity: 3
Enter Available Food Packets: 4
Enter Available Medical Kits: 4
Enter Number of Volunteers: 3
Camp added successfully!

SMART DISASTER RELIEF RESOURCE MANAGEMENT SYSTEM
1. Add Camp
2. Register Victim
3. Distribute Resources
4. View Records
5. Search Victim
6. Generate Report
7. Exit
Enter choice: 2
Enter Victim ID: 222
Enter Name: nuzhat
Enter Age: 22
Enter Health Condition (normal/critical): normal
Enter Camp ID to assign: 123
Victim registered successfully!

SMART DISASTER RELIEF RESOURCE MANAGEMENT SYSTEM
1. Add Camp
2. Register Victim
3. Distribute Resources
4. View Records
5. Search Victim
6. Generate Report
```

```
7. Exit
Enter choice: 3
Enter Victim ID: 222
Food packet distributed.
Medical kit not required.
SMART DISASTER RELIEF RESOURCE MANAGEMENT SYSTEM
1. Add Camp
2. Register Victim
3. Distribute Resources
4. View Records
5. Search Victim
6. Generate Report
7. Exit
Enter choice: 4

--- CAMPS ---
{'camp_id': '123', 'location': 'hnk', 'capacity': 3, 'food': 3, 'medical': 4, 'volunteers':
3, 'occupancy': 1}

--- VICTIMS ---
{'victim_id': '222', 'name': 'nuzhat', 'age': 22, 'health': 'normal', 'camp_id': '123', 'foo
d_received': 1, 'medical_received': 0}

SMART DISASTER RELIEF RESOURCE MANAGEMENT SYSTEM
1. Add Camp
2. Register Victim
3. Distribute Resources
4. View Records
5. Search Victim
6. Generate Report
7. Exit
Enter choice: 5
Enter Victim ID to search: 222
{'victim_id': '222', 'name': 'nuzhat', 'age': 22, 'health': 'normal', 'camp_id': '123', 'foo
d_received': 1, 'medical_received': 0}

SMART DISASTER RELIEF RESOURCE MANAGEMENT SYSTEM
1. Add Camp
2. Register Victim
3. Distribute Resources
4. View Records
```

Ln 439, Col 4 Spaces: 4 UTF-8 CRLF { } Python

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
4. View Records
5. Search Victim
6. Generate Report
7. Exit
Enter choice: 6

----- ANALYTICAL REPORT -----
Total Camps: 1
Total Victims Registered: 1
Camp with Highest Occupancy: 123
Total Food Packets Distributed: 1
Total Medical Kits Distributed: 0
Number of Critical Victims: 0
-------------------------------

SMART DISASTER RELIEF RESOURCE MANAGEMENT SYSTEM
1. Add Camp
2. Register Victim
3. Distribute Resources
4. View Records
5. Search Victim
6. Generate Report
7. Exit
Enter choice: 7
Exiting system...
Top element: 20
Popped: 20
Is empty? False
Linear Search Result: 9998
Binary Search Result: 9998
Front: 10
Dequeued: 10
Bubble Sort: [11, 12, 22, 25, 64]
Selection Sort: [11, 12, 22, 25, 64]
PS C:\Users\hp\Desktop\AIAC>
```